

Greedy Algorithms

Kruskal's Algorithm

Adjacency list of pairs vertex: (vertex, weight)

The approach:

1. Sort the edges by weight (in increasing order)
2. $\forall e$ in this order, if e does not create a cycle add it to X
(start with 0 edges)
3. Output $T = X$

Correctness:

First step:

$\emptyset \cup \{e_1\} \rightarrow$ The cheapest edge is part of some MST!

Inductive step (i^{th} step):

$$X \cup \{e_i\} = X'$$

Can represent more than one connected components (CC) on either sides of the cut

$|V| - 1$

Can connect two already formed CCs (or) one CC and one singleton (or) two singletons. Will never connect 2 nodes in 1 CC (cycle!). Therefore, e_i goes across cut & e_i is the cheapest outside X that does not create a cycle.

Time complexity:

Step 1: $O(|E| \log |E|)$

Step 2: $O(|E|(|V| + |E|))$



Quadratic in # of edges because we are running DFS.

OK, but not great!

DFS revealing back-edges in undirected graph also means there is a cycle (2 different paths between same nodes).

But the DFS is possibly doing a lot of redundant things every time it is run, as the graph only changes by an edge every iteration!

A way out: Use connected component labels on the go

A faster way to do that is using a special data structure: Union Find

Faster Kruskal:

of CCs at the start is $|V|$

- Maintain the connected components $CC(v)$ along the way.
- For the next cheapest edge $e = (u, v)$, check whether $CC(u) = CC(v)$ and then add only when not equal.
- After adding e update the 2 connected components (label) by merging them into 1.

procedure `kruskal`(G, w)

Input: A connected undirected graph $G = (V, E)$ with edge weights w_e

Output: A minimum spanning tree defined by the edges X

for all $u \in V$:

`makeset`(u) $\longrightarrow$ `makeset`(x): create a singleton set containing just x .

$|V|$ `makeset` $O(|V|)$

$X = \{\}$

Sort the edges E by weight $O(|E| \log |E|) \leq O(|E| \log |V|^2) = O(|E| \log |V|)$

for all edges $\{u, v\} \in E$, in increasing order of weight:

if `find`(u) $\neq$ `find`(v): $\longrightarrow$ `find`(x): to which set does x belong? $|E|$ `find`

add edge $\{u, v\}$ to X

$O(\log |V|)$

`union`(u, v) $\longrightarrow$ `union`(x, y): merge the sets containing x and y .

Return X

$|V| - 1$ `union` $O(\log |V|)$

The functions

Maintain a collection C of disjoint subsets of $\{1, 2, \dots, n\}$

1. $\text{makeset}(x): C \leftarrow C \cup \{x\}$
2. $\text{find}(x)$: return “name” of S_x (representative element / label)
3. $\text{union}(x, y)$: remove S_x, S_y from C and $C \leftarrow C \cup \{S_x \cup S_y\}$

Naïve: Array- $A[i]$ stores the name of vertex i (S_i)

Find: $O(1)$, Union: $O(n)$, Makeset: $O(n)$



Need to scan the array and replace one particular name everywhere!

Kruskal's: $O(m \log m) + O(n) + O(m) + O(n^2)$



Dense: $O(n^2 \log n)$

>

$O(n^2)$

Sparse: $O(n \log n)$

<

$O(n^2)$

Prim's Algorithm

From the cut property:

Any algorithm conforming to the following greedy schema is guaranteed to work

$X = \{ \}$ (edges picked so far)

repeat until $|X| = |V| - 1$:

pick a set $S \subset V$ for which X has no edges between S and $V - S$

let $e \in E$ be the minimum-weight edge between S and $V - S$

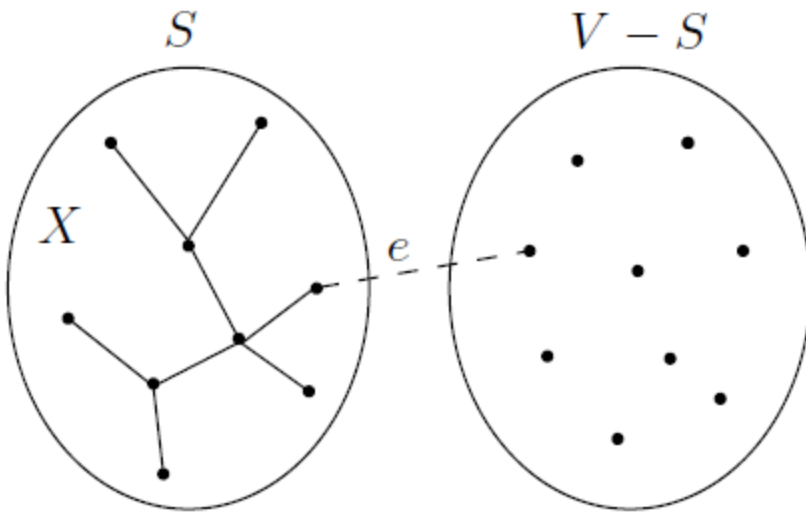
$X = X \cup \{e\}$

In Prim's

- X always forms a subtree of an MST.
- S is chosen as the set of X 's vertices.
- X grows into $X \leftarrow X \cup \{e\}$, where e is the lightest edge between a node in S and a node outside S .

Equivalently: S grows by including the vertex $v \notin S$:

$$\text{cost}(v) = \min_{u \in S} w(u, v).$$



Very similar in procedure
(not purpose) to Dijkstra's!



Same run time based on
priority queue!

procedure prim(G, w)

Input: A connected undirected graph $G = (V, E)$ with edge weights w_e

Output: A minimum spanning tree defined by the array prev

for all $u \in V$:

$\text{cost}(u) = \infty$

 prev(u) = nil

Pick any initial node u_0

$\text{cost}(u_0) = 0$

⋮

```

H = makequeue(V)  $O(n)$  (priority queue, using cost-values as keys)
while H is not empty:
    v = deletemin(H)  $O(n)$ 
    for each  $\{v, z\} \in E$ :  $\rightarrow$  Edge weights!
        if cost(z) > w(v, z):
            cost(z) = w(v, z)
            prev(z) = v
            decreasekey(H, z)  $O(1)$ 

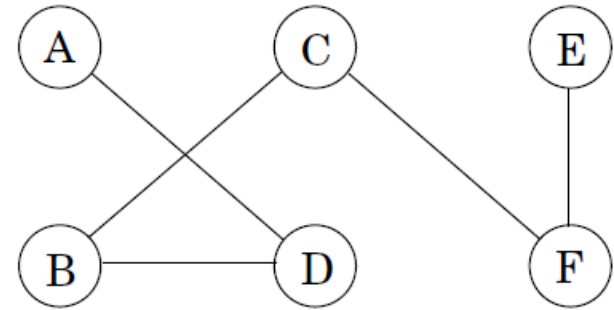
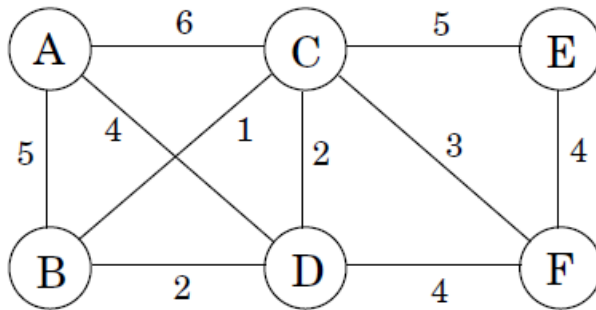
```

Do you need any other checks here?? *Yes*

An example:

$O(n^2)$

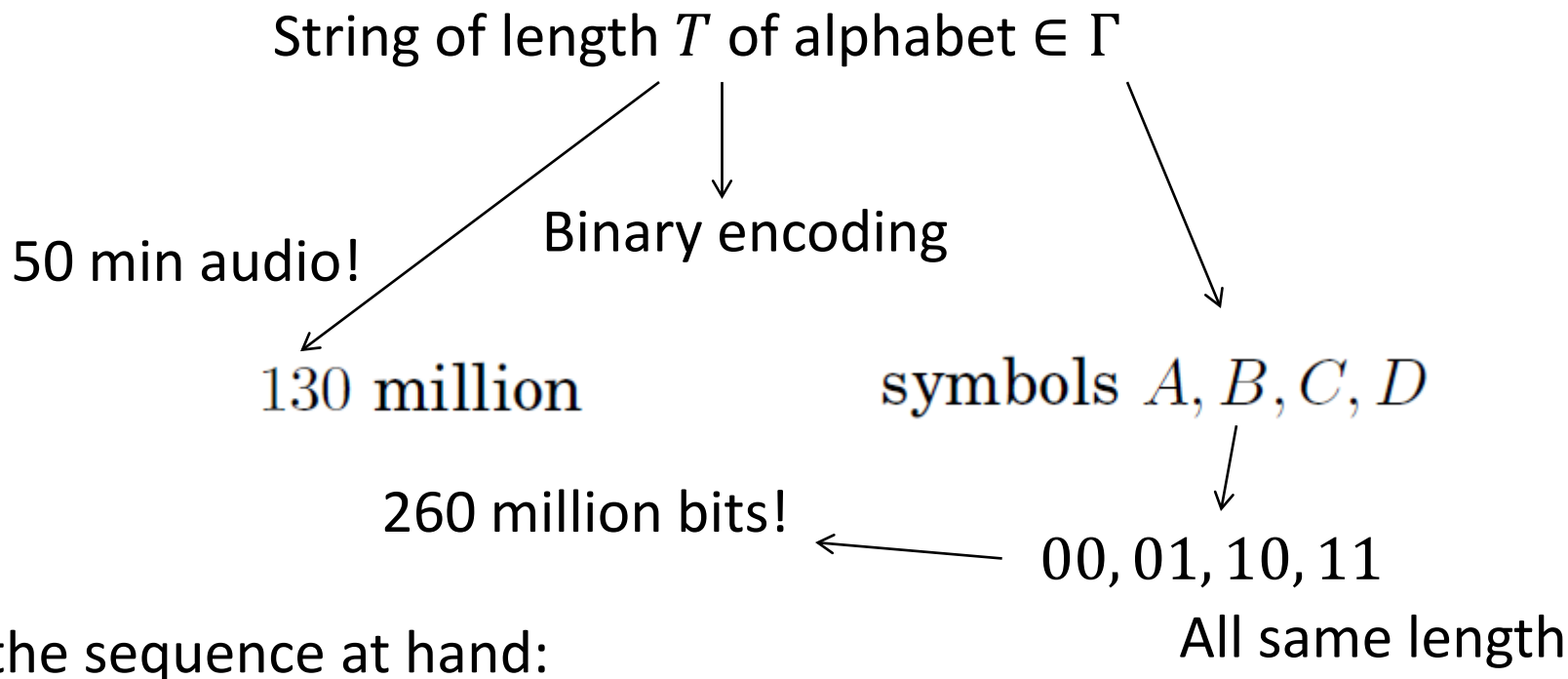
Binary heap $O(m \log n)$



Set S	A	B	C	D	E	F
$\{\}$	0/nil	∞ /nil	∞ /nil	∞ /nil	∞ /nil	∞ /nil
A		5/ A	6/ A	4/ A	∞ /nil	∞ /nil
A, D		2/ D	2/ D		∞ /nil	4/ D
A, D, B			1/ B		∞ /nil	4/ D
A, D, B, C					5/ C	3/ C
A, D, B, C, F					4/ F	

Huffman Encoding

Sampling and Quantization leads to:



In the sequence at hand:

Symbol	Frequency
A	70 million
B	3 million
C	20 million
D	37 million

variable-length encoding

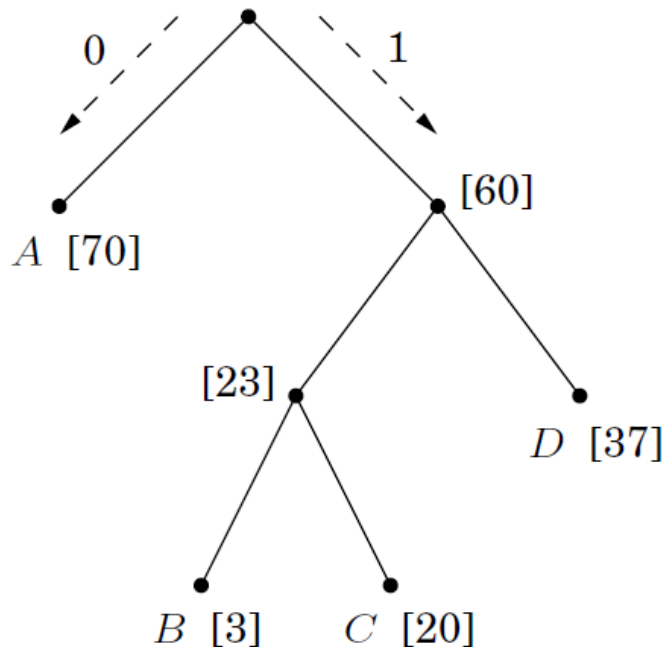
But decoding?

{0, 01, 11, 001}

Prefix-free property: no codeword can be a prefix of another codeword

Full Binary Tree with **Symbols at the Leaves!**

Left – 0, Right – 1



Symbol	Codeword
<i>A</i>	0
<i>B</i>	100
<i>C</i>	101
<i>D</i>	11

Decoding Algo

(for every bit, one
check-one traversal)

But how to form the tree?

n symbols $\longrightarrow f_1, f_2, \dots, f_n$

$$\text{cost of tree} = \sum_{i=1}^n f_i \cdot (\text{depth of } i\text{th symbol in tree})$$



Number of bits required!

The cost of a tree is the sum of the frequencies of all leaves and internal nodes, except the root.

The two symbols with the smallest frequencies must be at the bottom of the optimal tree.

A Greedy Approach to Construct the Tree!

Correctness / Optimality: No intermediate node formed can have a lower frequency associated compared to the earlier (higher / equal depth) intermediate nodes formed and leaf nodes assigned symbols.

procedure Huffman(f)

Input: An array $f[1 \cdots n]$ of frequencies

Output: An encoding tree with n leaves

let H be a priority queue of integers, ordered by f

for $i = 1$ to n : insert(H, i)

for $k = n + 1$ to $2n - 1$:

$i = \text{deletemin}(H), j = \text{deletemin}(H)$

★ create a node numbered k with children i, j

$f[k] = f[i] + f[j]$

insert(H, k)

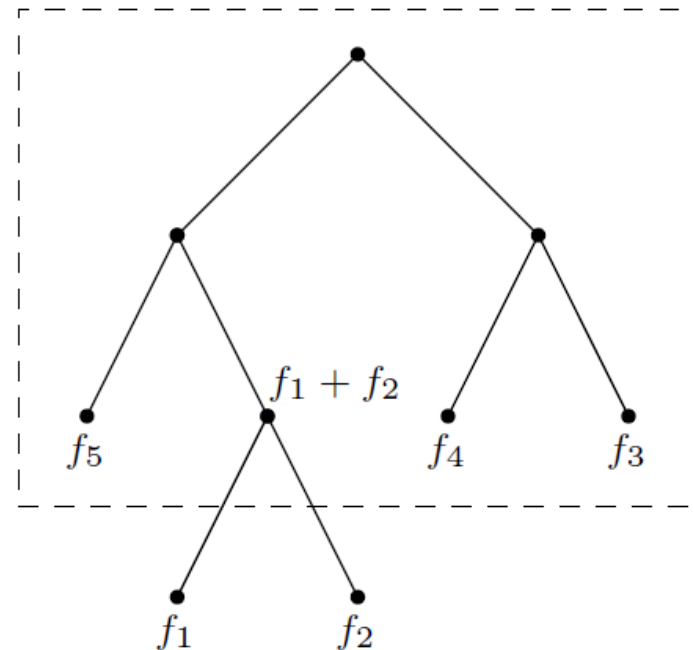
—1 size every iteration

$O(n^2)$ with Array

$O(n \log(n))$ with Binary heap

★ → Form adjacency list (tree)

of nodes: $2n - 1$



Dynamic Programming

Consider Fibonacci

That's Dynamic Programming!

```
function fib1(n)
if n=0: return 0
if n=1: return 1
return fib1(n-1) + fib1(n-2)
```

↓

```
function fib1fast(n)
create an array  $f[0 \dots n]$ 
return fibmemo(n, f)
```

Recursive

Memoization

```
function fibmemo(n, f)
if  $n \leq 1$ : return  $n$ 
elif  $f[n] \neq []$ : return  $f[n]$ 
else:
 $f[n] = \text{fibmemo}(n-1, f) + \text{fibmemo}(n-2, f)$ 
return  $f[n]$ 
```

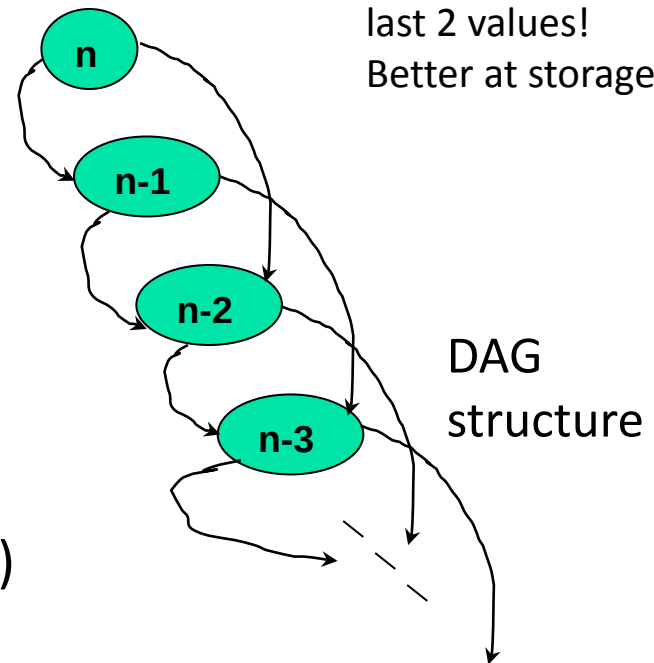
Top-down

Iterative

```
function fib2(n)
create an array  $f[0 \dots n]$ 
 $f[0] = 0, f[1] = 1$ 
for  $i = 2 \dots n$ :
 $f[i] = f[i-1] + f[i-2]$ 
return  $f[n]$ 
```

Bottom-up

I can just store
last 2 values!
Better at storage?



Always being
picked from
memory!

- * Top can call anybody from bottom as per connection
- * Bottom supplies to all connected to the top